Simulating Stellar Cluster Evolution in Parallel

TEAM: Hasita Kanamarlapudi, Jenny Tao
WEBSITE:

SUMMARY

We plan to accelerate star cluster evolution simulation through parallelism on CPUs (OpenMP) and GPUs (CUDA), and evaluate their performances compared to a single-thread baseline. We target a star cluster of 100k to 1M stars where gravity moves the stars and each star ages, changing temperature and mass.

BACKGROUND

Star cluster simulation is a compute-intensive application that models the following two processes simultaneously:

1. Gravitational forces: Every pair of stars in outer space attract each other by their mutual gravity. Given one million stars, this means half a trillion unique gravitational interactions. Because the gravity calculations for different stars at a given timestep are independent, stars can be handled by groups and parallelized across many processors to compute at the same time.
2. Stellar lifespan updates: Stars start on the main sequence, evolve into red giants and supergiants, and end as white dwarfs, neutron stars, or black holes. Because the aging of a star depends solely on its own properties like mass and temperature, we can calculate its lifespan updates in parallel and update its mass and color individually.

CHALLENGES

DEPENDENCIES:

- Timesteps are sequential since the positions, sizes, and masses of stars in step t influence step t+1. So parallelism must be completely within a step. Physics calculations per star are independent, so this allows good parallelism.
- If we implement granularity in timesteps based on cluster density, then fast-stepping core stars will need information from slow-stepping outer stars. We would need to decide if we allow some level of staleness or calculate partial results.
- Barnes-Hut tree building to calculate force effects will require some kind of synchronization or lock-free data structure.

- Barrier between each of the following phases: tree building, force calculation, and position updating. We can't calculate forces until the tree is built, and position can't be updated before the force effects are computed. Stellar evolution doesn't impact computations in this timestep, but must be done before the next timestep's tree build, so we can run this in parallel with the previous three phases.

WORKLOAD:

- Direct sum is very compute-intensive, while Barnes-Hut provides a faster approximation. Barnes-Hut, however, means that memory can be the bottleneck as we chase pointers. We can try to sort stars spatially to promote locality. Evolution requires only per-star data so fully local.
- On a single GPU, there is communication between CPU and GPU and synchronization between threads. For multi-GPU, communication occurs at each timestep since tree data is replicated or split across devices. For replication, all-gather must occur for each step. For split, computing will require data requests. Stars crossing machine boundaries must be moved to GPU owning that region.
- Gravity pulls heavy stars into the core over time, so load balancing must be dynamic.

DIVERGENCE:

- Core stars have more node dependencies than outskirt stars.
- Different timestep size per location can mean idle threads at boundaries.
- Evolution requires different formulas per life phase.

SYSTEM MAPPING CONSTRAINTS:

- Some threads in warp will be idle because of divergence in star life-phase and tree traversal path.
- Tree structure might not work effectively with GPU, so restructuring may be necessary.
- With small time-steps near core, GPU may be underutilized if only a fraction of stars are updated.

RESOURCES

- GHC lab machines, NVIDIA GPUS for CUDA, and multicore CPUs for OpenMP comparison. We don't anticipate needing PSC unless we explore multi-GPU performance.
- Stellar evolution formulas by potentially using published SSE code.
- Barnes-Hut paper, any SSE papers as needed.

GOALS AND DELIVERABLES

PLAN TO ACHIEVE:

- Serial simulator
- Comparison between OpenMP and CUDA using parallel direct-sum gravity
- Stellar evolution, where mass loss affects gravity calculations
- GPU Barnes-Hut reaching ~1M stars
- Optimize memory layout (e.g. spatial sorting of stars)
- Visualizer of star cluster and HR diagram of stars in their various stages of life
- Benchmarks and poster

Star clusters contain anywhere from 100k to a few million stars so 1M stars represents a cluster somewhat accurately and fits easily in 1 GPU's memory. Since we are worried about idle threads and underutilization, we want as many stars and active threads as possible. Stellar evolution and direct-sum gravity can be completed by each of us separately since we combine them in Week 3. Load balancing will be the main focus.

HOPE TO ACHIEVE:

- Timestep granularity: finer steps for core and coarser for outer stars
- Use CPU for stellar evolution at the same time as GPU gravity computation.
- Multi-GPU scaling??

Timestep granularity can help as an optimizing factor if we find that outer stars are not really changing per timestep. Grouping by timestep size can help us address divergence directly. CPU stellar evolution should not be conceptually difficult and may handle branching better, but will require some care in optimizing copying over data. Multi-GPU scaling is ambitious and we don't believe we will get to it, but if we do, we will choose tree replication over splitting to avoid dealing with stars that move across per-machine region boundaries.

IF WORK GOES SLOWLY, we will submit a direct-sum CUDA with coupled stellar evolution and visuals, with whatever Barnes-Hut we can get. This still shows parallel speedup and comparison to CPU OpenMP.

DEMO: The focus of the project is on parallelism and not visualization, but we aim to make a simple HR diagram and visualizer of some sampling of stars to show the impact of stellar evolution and gravity. The visualizer would make differences in time-step granularity clearer.

PLATFORM

We will use the NVIDIA GPUs and multicore CPUs on the GHC machines, programming in C++ with CUDA and OpenMP. The workload is highly data-parallel: within each step, every star's force can be computed independently, so 1M stars map naturally onto 1M GPU threads. 1M stars will also fit easily in GPU memory, so we can avoid slow CPU-GPU transfers. Gravity computation is mainly floating-point arithmetic, and GPUs offer far more throughput than CPUs. However, since Barnes-Hut traversal is irregular, this can cause warp divergence and scattered memory access, and stellar evolution is branched code better suited to CPUs. This is why we are

comparing both CPU and GPU versions of gravity and stellar evolution. CUDA gives us direct control over thread blocks, shared memory, and synchronization for tree building, and an OpenMP version provides a fair CPU baseline. PSC resources would only be needed for the stretch goal of multi-GPU scaling.

SCHEDULE

Week 1: 11/9-11/15	Serial simulation, initial conditions, evolution formulas set up
Week 2: 11/16-11/22	Finish direct-sum gravity and compare OpenMP with CUDA, finish evolution, basic visualizer, milestone report due 11/23
Week 3: 11/23-11/29	Barnes-Hut tree-build and traversal on GPU, combine evolution and tree
Week 4: 11/30-12/6	Optimize Barnes-Hut, explore timestep granularity
Week 5: 12/7-12/11	Final benchmarks, visualizer polishing for demo, report, poster